



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN DE SERVICIOS Y
PROCESOS**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PSP03 Ejercicio 2

Presentado por: Eugen Moga

Fecha 27/01/2026

ÍNDICE

Servidor UDP	1
Cliente UDP.....	4
Prueba de funcionamiento	6

Servidor UDP

Este servidor va a tomar el rol de receptor para recibir los mensajes que envíe el cliente.

Primero compruebo que no se insertan argumentos como parámetros la ejecución del servidor UDP tiene que quedar de esta forma:

```
java -jar ServidorUDP.jar
```

```
9  /**
10  *
11  * @author Eugen Moga
12  *
13  * PSP Tema 3 Tarea
14  * Ejercicio 2: Receptor de datagramas con socket UDP
15  */
16  public class ServidorUDP {
17
18      public static void main(String[] args) {
19
20          // Sin argumentos, comprueba que no se ingresa ningun argumento
21          if (args.length != 0){
22              System.err.println("Uso: java -jar ServidorUDP.jar");
23              return;
24          }
25      }
```

Creo la variable PUERTO para indicar poder configurar el puerto donde el servidor UDP permanecerá a la escucha

```
26
27      // Se indica el puerto donde el servidor UDP permanecera a la escucha
28      final int PUERTO = 1500;
```

Creo el socket UDP con DatagramSocket y le asigno el puerto mediante la variable creada anteriormente.

También creo un BufferedWriter para poder escribir los mensajes recibidos en el archivo "mensaje.txt" creado con FileWriter

Después muestro el mensaje "Esperando mensajes en el puerto" + puerto

```
29
30 // Se crea el socket UDP y se indica el puerto en el que escucha
31 // Se crea BufferedWriter para almacenar los mensajes recibidos
32 // Utilizo try-catch para asegurar el cierre de recursos
33 try (DatagramSocket sSocket = new DatagramSocket(PUERTO);
34      BufferedWriter bw = new BufferedWriter(new FileWriter("mensaje.txt"))){
35
36     System.out.println("Esperando mensajes en el puerto " + PUERTO);
37
```

Declaro la variable cadena de tipo Byte para almacenar los datos recibidos

```
37
38 // Buffer de bytes para almacenar datos recibidos
39 byte [] cadena = new byte[1024];
40
```

Creo el datagrama paquete con DatagramPacket que recibe la información y lo guardo en el array de bytes cadena.

Luego pongo el servidor a la espera de recibir un paquete

```
43
44 // Se crea el datagrama que recibe la informacion
45 DatagramPacket paquete = new DatagramPacket(cadena, cadena.length);
46
47 // El servidor queda bloqueado hasta recibir un datagrama
48 sSocket.receive(paquete);
49
```

Convierto el datagrama recibido a una cadena String mensaje para ello le paso 3 argumentos del constructor String: que coja los datos del paquete con paquete.getData(), que empiece en la posición 0, y que solo coja la longitud del paquete con paquete.getLength(), de esta forma solo coje los bytes que tiene el mensaje y no los 1024 que ocupa la cadena

```
49
50 // Convierte el contenido del datagrama a string
51 String mensaje = new String(paquete.getData(), 0, paquete.getLength());
52
```

Compruebo que no ha recibido la cadena "FIN" y con el BufferedWriter escribo el mensaje en el fichero.

```
53 // Si recibe la cadena FIN se finaliza la ejecucion.
54 if (mensaje.equals("FIN")){
55     System.out.println("Mensaje FIN recibido. Cerrando servidor.");
56     break;
57 }
58
59 // Se escribe el mensaje recibido en el fichero
60 bw.write(mensaje);
61 bw.newLine();
62 }
63 System.out.println("Mensajes guardados correctamente.");
64
```

Para finalizar cierro el catch que controla las excepciones con el socket

```
64
65 }catch (Exception e){
66     // Se capturan errores relacionados con el sockets
67     System.out.println("Error servidor UDP: " + e.getMessage());
68 }
```

Cliente UDP

Para el cliente UDP solicito que se introduzca 2 argumentos como parámetro, para indicar la ip del servidor y el puerto donde escucha. Con if hago la comprobación que se han introducido 2 argumentos

```
8  /**
9  * @author Eugen Moga
10 * PSP Tema 3 Tarea
11 * Ejercicio 2: Cliente UDP para el envio de datagramas
12 */
13 public class ClienteUDP {
14
15     /**
16     * @param args argumentos de linea de comandos
17     *      args[0] -> IP del servidor
18     *      args[1] -> Puerto del servidor
19     */
20     public static void main(String[] args) {
21
22         // Se comprueba el numero de argumentos recibidos
23         // Tiene que ser dos IP y Puerto
24         if (args.length != 2) {
25             System.err.println("Uso: java -jar ClienteUDP.jar <ip_servidor> <puerto>")
26             return;
27         }
28     }
```

Asigno el primer argumento a la variable ipServidor y el segundo argumento con la variable puerto, que es donde escucha el servidor

```
28
29     // Se asignan los parametros introducidos a las variables
30     String ipServidor = args[0];
31     int puerto = Integer.parseInt(args[1]);
32
```

Creo el socket UDP con DatagramSocket. Esto dentro de un bloque try-catch para controlar el cierre automático de recursos.

```
32
33     // Creacion del socket UDP del cliente
34     try {DatagramSocket socket = new DatagramSocket();}
35
```

El socket UDP no puede usar una variable de tipo String como ipServidor para indicar la dirección ip del servidor. Necesita un objeto de tipo InetAddress por eso valido la dirección ip en esta linea

```
35
36 // Obtencion de la direccion IP del servidor
37 InetAddress direccion = InetAddress.getByNome(ipServidor);
38
```

Utilizo un bucle for para iterar hasta el numero 10000 y generar los mensajes necesarios.

1. Creo una variable de tipo String: mensaje y en cada iteración del bucle for le asigno el valor de "Mensaje: " + i
2. Creo la variable cadena de tipo byte y le asigno el valor en bytes de la variable mensaje. Para poder enviar el datagrama.
3. Cero el datagrama con DatagramPacket y le indico que envíe la variable cadena, la longitud de la cadena a la dirección ip, y al puerto ya configurados anteriormente.
4. Con socket.send(paquete) le indico que envíe el paquete.

```
38
39 // Bucle para enviar 10000 mensajes numerados al servidor
40 for(int i = 1; i <= 10000; i++){
41     String mensaje = "Mensaje: " + i;1
42     byte[] cadena = mensaje.getBytes();2
43
44     // Se crea el datagrama con el mensaje, la IP y el puerto
45     DatagramPacket paquete = 3
46         new DatagramPacket(cadena, cadena.length, direccion, puerto);
47
48     // Se envia el datagrama
49     socket.send(paquete);4
50 }
```

Al terminar la iteración de los 1000 mensajes envió el mensaje "FIN" y muestro en pantalla "Mensajes enviados correctamente"

```
51
52 // Se envia FIN para indicar el final de la transmision
53 byte[] fin = "FIN".getBytes();
54 DatagramPacket paqueteFin =
55     new DatagramPacket(fin, fin.length, direccion, puerto);
56
57 socket.send(paqueteFin);
58
59 System.out.println("Mensajes enviados correctamente.");
60
```

Para finalizar cierro el bloque try-catch para capturar errores relacionados con la comunicación UDP

```
60  
61 } catch (Exception e) {  
62     // Se capturan errores relacionados con la comunicacion UDP  
63     System.out.println("Error cliente UDP: " + e.getMessage());  
64 }  
65
```

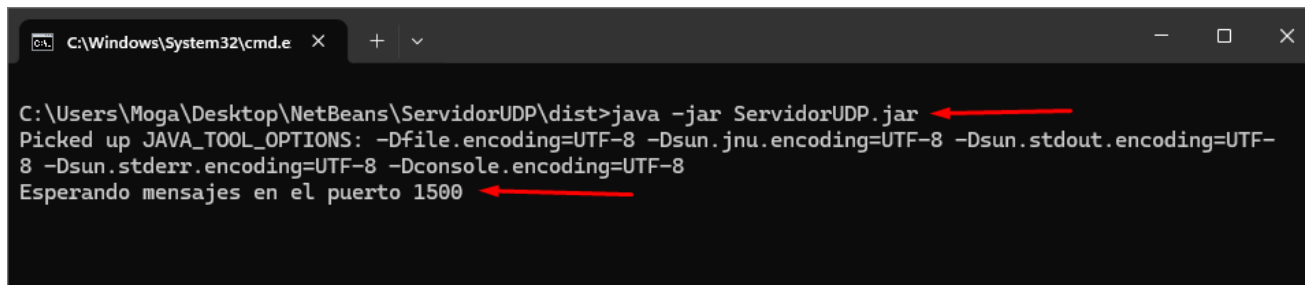
Prueba de funcionamiento

Para probar el correcto funcionamiento compilo tanto el proyecto ClienteUDP como ServidorUDP esto me genera los archivos ClienteUDP.jar y ServidorUDP.jar respectivamente.

Ejecuto primero el Servidor desde linea de comandos con:

```
java -jar ServidorUDP.jar
```

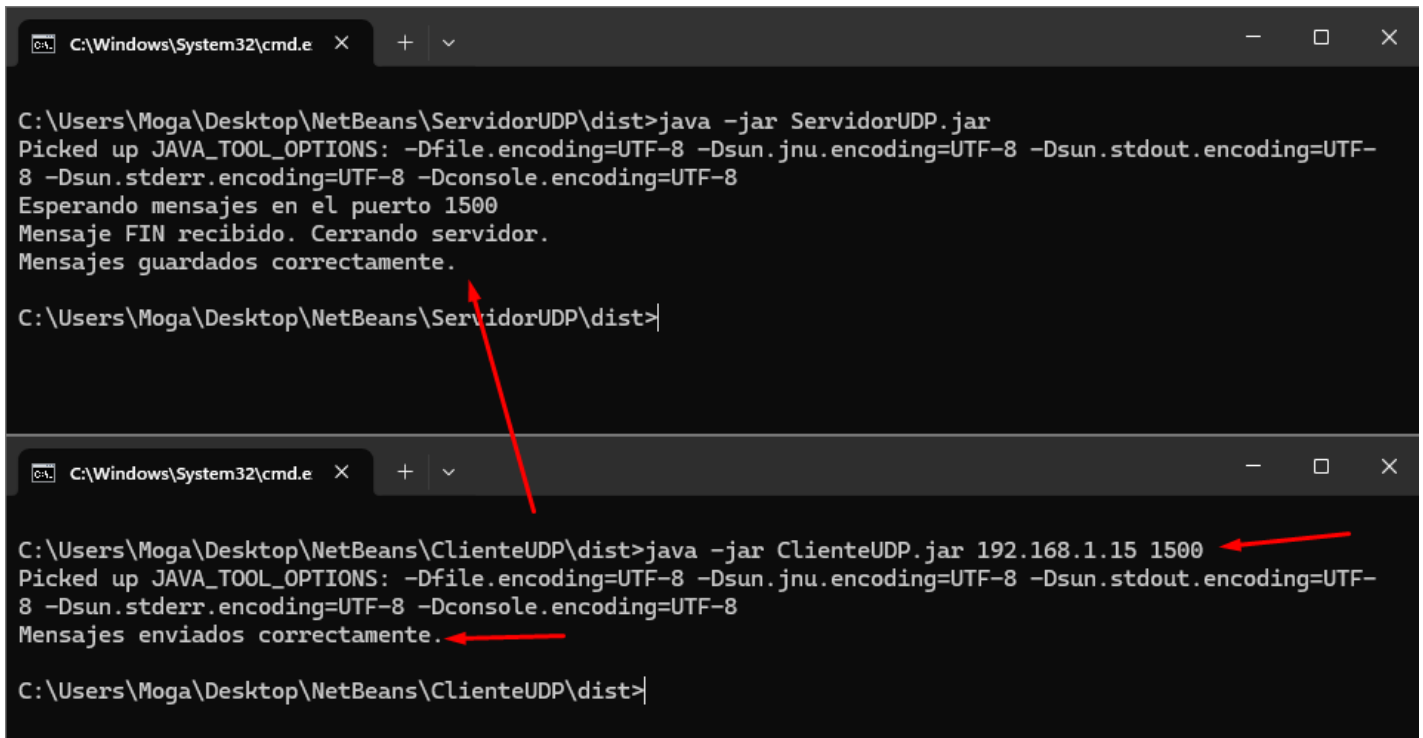
Se puede ver en la captura como el servidor muestra el mensaje: Esperando mensajes en el puerto 1500



```
C:\Windows\System32\cmd.e  X  +  v  -  □  X  
C:\Users\Moga\Desktop\NetBeans\ServidorUDP\dist>java -jar ServidorUDP.jar  
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8  
Esperando mensajes en el puerto 1500
```


Paso a ejecutar el ClienteUDP.jar con el comando:

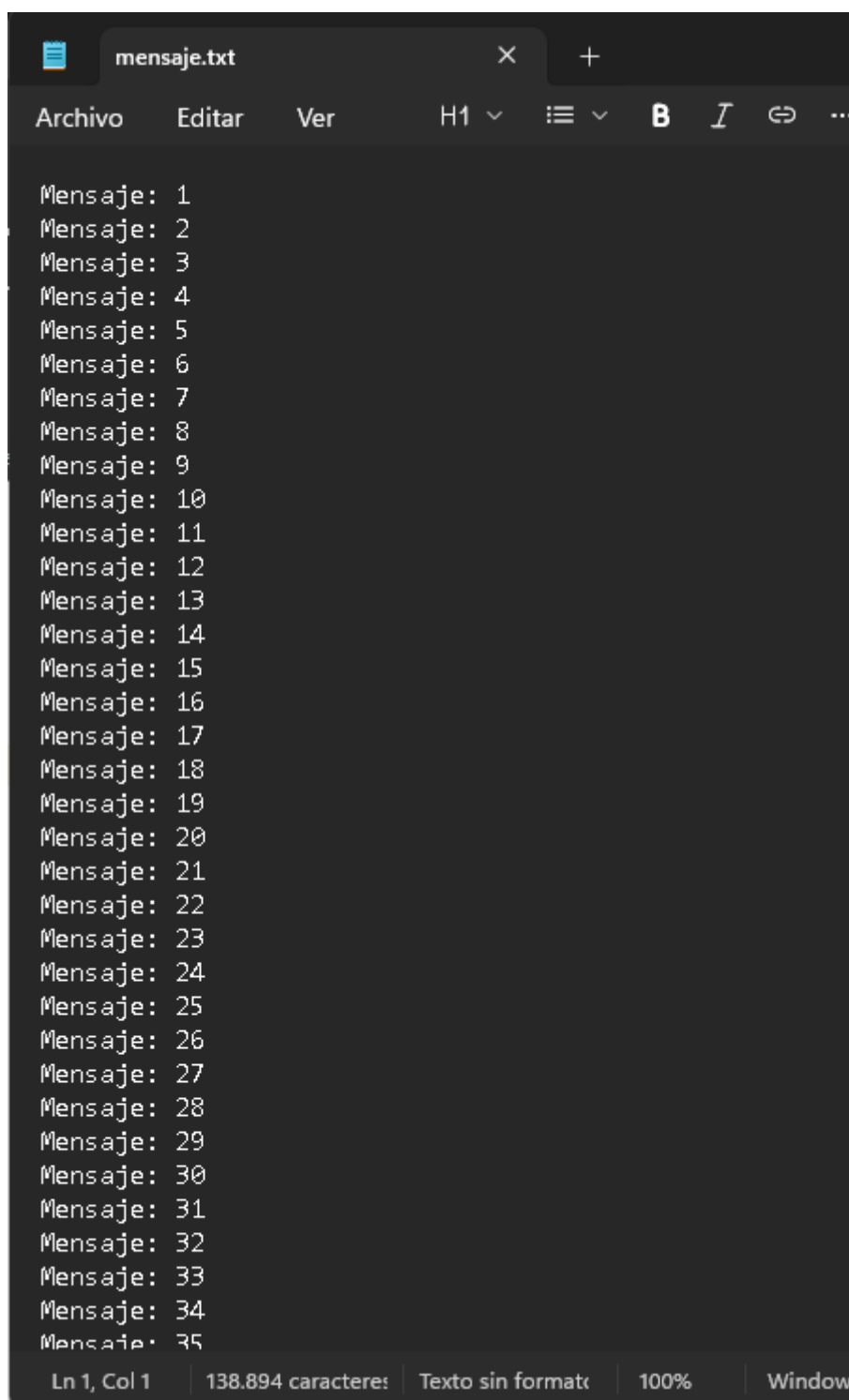
```
java -jar ClienteUDP.jar 192.168.1.15 1500
```



```
C:\Windows\System32\cmd.e X + v
C:\Users\Moga\Desktop\NetBeans\ServidorUDP\dist>java -jar ServidorUDP.jar
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Esperando mensajes en el puerto 1500
Mensaje FIN recibido. Cerrando servidor.
Mensajes guardados correctamente.
C:\Users\Moga\Desktop\NetBeans\ServidorUDP\dist>

C:\Windows\System32\cmd.e X + v
C:\Users\Moga\Desktop\NetBeans\ClienteUDP\dist>java -jar ClienteUDP.jar 192.168.1.15 1500
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Mensajes enviados correctamente.
C:\Users\Moga\Desktop\NetBeans\ClienteUDP\dist>
```

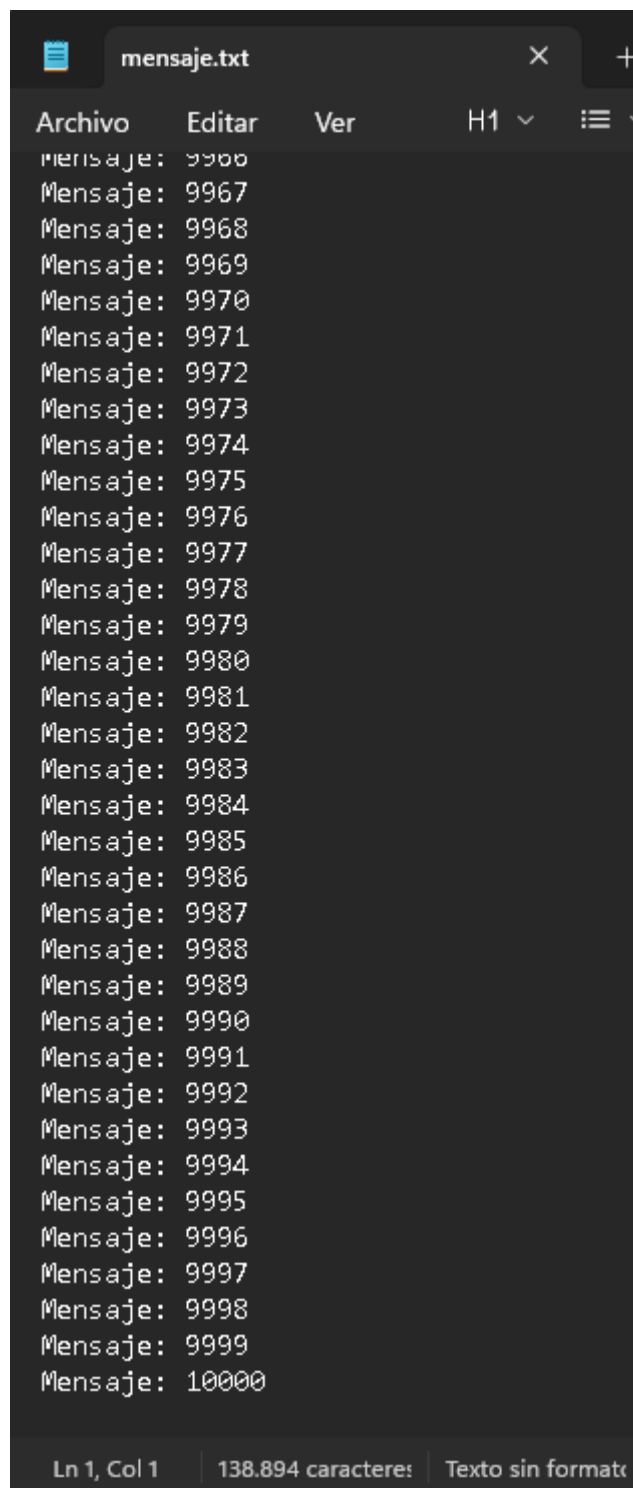
Ahora abro el archivo mensaje.txt y puedo ver que los mensajes han llegado en orden



The image shows a screenshot of a text editor window with a dark theme. The title bar at the top indicates the file is 'mensaje.txt'. The menu bar includes 'Archivo', 'Editar', and 'Ver'. The toolbar shows 'H1', a list icon, bold (B), italic (I), and a link icon. The main text area contains a list of 35 messages, each starting with 'Mensaje: ' followed by a number from 1 to 35. The status bar at the bottom shows 'Ln 1, Col 1', '138.894 caractere:', 'Texto sin formateo', '100%', and 'Window'.

```
Mensaje: 1
Mensaje: 2
Mensaje: 3
Mensaje: 4
Mensaje: 5
Mensaje: 6
Mensaje: 7
Mensaje: 8
Mensaje: 9
Mensaje: 10
Mensaje: 11
Mensaje: 12
Mensaje: 13
Mensaje: 14
Mensaje: 15
Mensaje: 16
Mensaje: 17
Mensaje: 18
Mensaje: 19
Mensaje: 20
Mensaje: 21
Mensaje: 22
Mensaje: 23
Mensaje: 24
Mensaje: 25
Mensaje: 26
Mensaje: 27
Mensaje: 28
Mensaje: 29
Mensaje: 30
Mensaje: 31
Mensaje: 32
Mensaje: 33
Mensaje: 34
Mensaje: 35
```

Ln 1, Col 1 | 138.894 caractere: | Texto sin formateo | 100% | Window

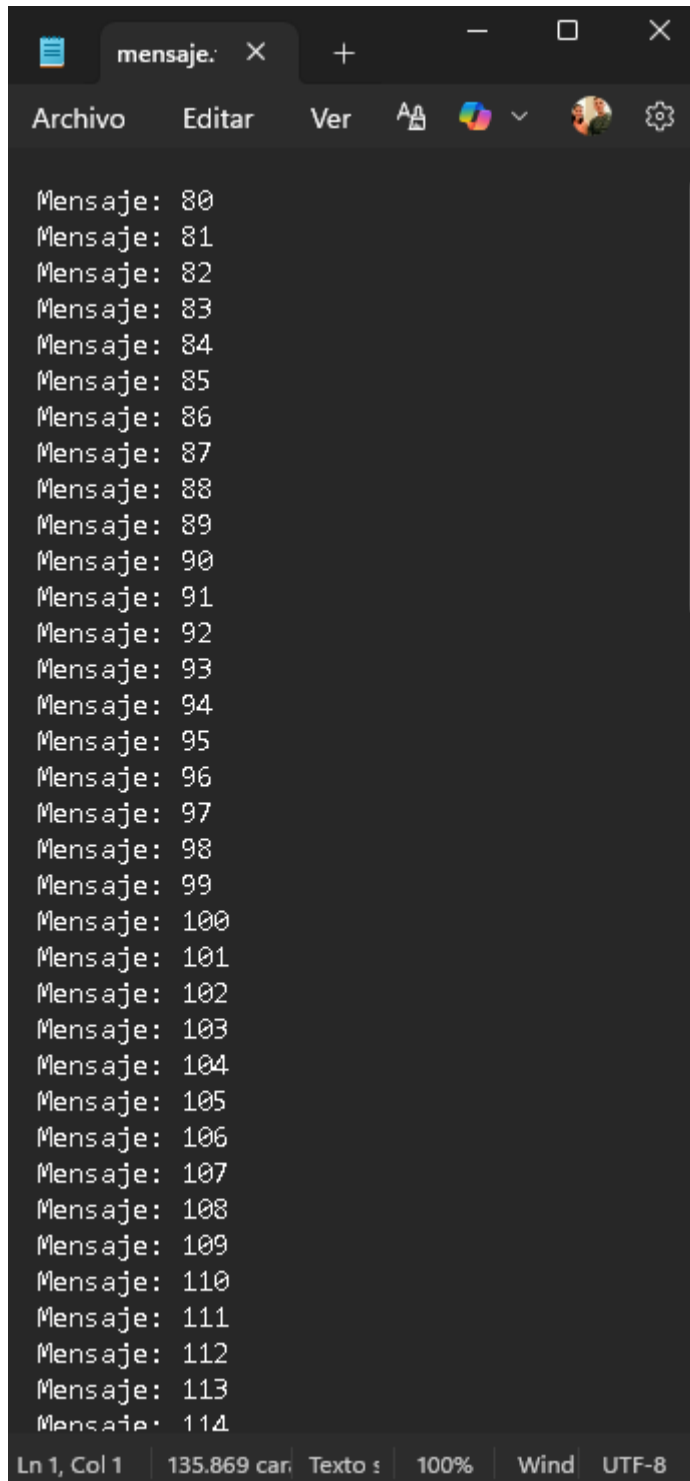


The image shows a code editor window with a dark theme. The title bar at the top reads 'mensaje.txt'. Below the title bar is a menu bar with the following items: 'Archivo', 'Editar', 'Ver', 'H1', and a hamburger menu icon. The main area of the editor contains a list of messages, each starting with 'Mensaje:'. The messages are numbered from 9967 to 10000, with the first line being 'Mensaje: 9967' and the last line being 'Mensaje: 10000'. The status bar at the bottom of the editor shows 'Ln 1, Col 1', '138.894 caracteres', and 'Texto sin formato'.

```
mensaje.txt
Archivo  Editar  Ver  H1  ≡
Mensaje: 9967
Mensaje: 9968
Mensaje: 9969
Mensaje: 9970
Mensaje: 9971
Mensaje: 9972
Mensaje: 9973
Mensaje: 9974
Mensaje: 9975
Mensaje: 9976
Mensaje: 9977
Mensaje: 9978
Mensaje: 9979
Mensaje: 9980
Mensaje: 9981
Mensaje: 9982
Mensaje: 9983
Mensaje: 9984
Mensaje: 9985
Mensaje: 9986
Mensaje: 9987
Mensaje: 9988
Mensaje: 9989
Mensaje: 9990
Mensaje: 9991
Mensaje: 9992
Mensaje: 9993
Mensaje: 9994
Mensaje: 9995
Mensaje: 9996
Mensaje: 9997
Mensaje: 9998
Mensaje: 9999
Mensaje: 10000
Ln 1, Col 1 | 138.894 caracteres | Texto sin formato
```

Voy a probar ejecutando Cliente udp desde otro ordenador que tengo en casa.

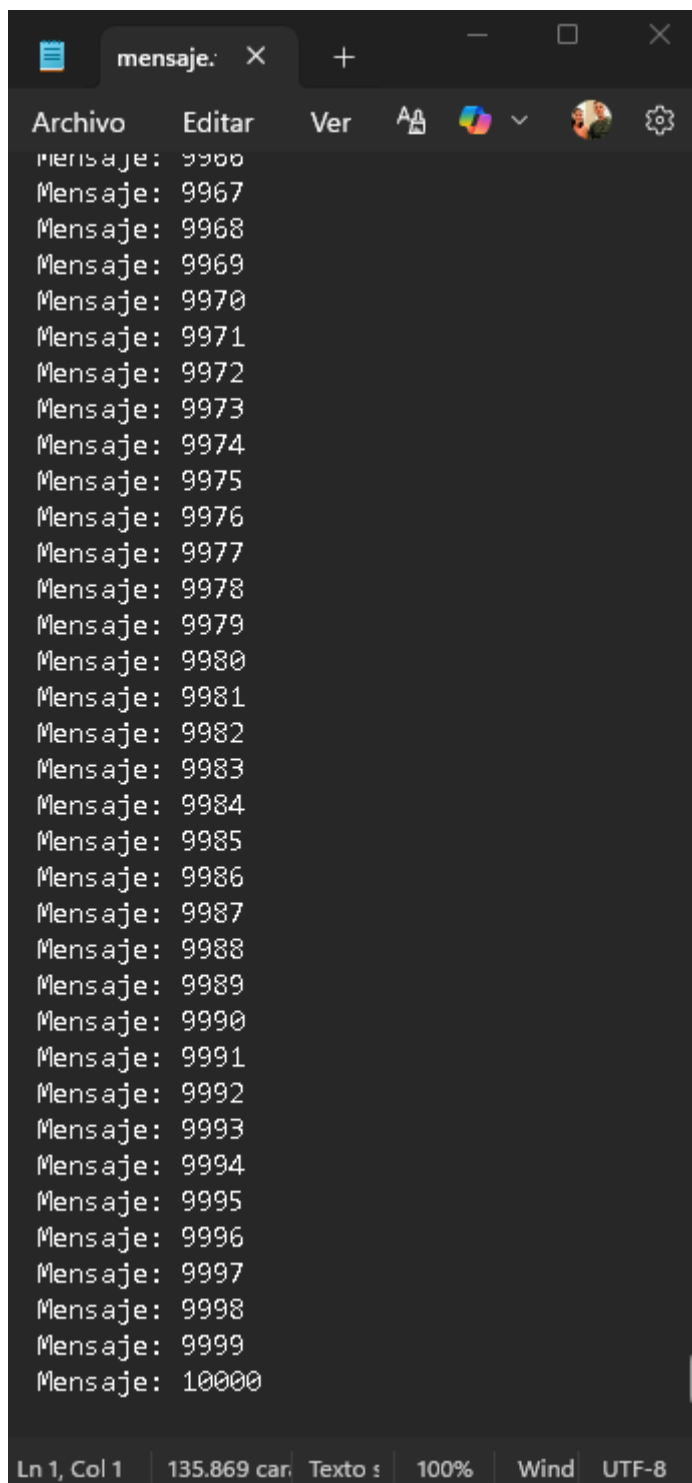
Puedo observar que el archivo txt empieza con el mensaje 80



The image shows a screenshot of a text editor window. The title bar at the top indicates the file is named 'mensaje.txt'. The menu bar includes 'Archivo', 'Editar', 'Ver', and icons for font settings, color selection, and window management. The main text area contains a list of messages, each starting with 'Mensaje: ' followed by a number. The numbers range from 80 to 114, with some numbers appearing to be repeated or slightly off (e.g., 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114). The status bar at the bottom shows 'Ln 1, Col 1', '135.869 car', 'Texto s', '100%', 'Wind', and 'UTF-8'.

```
Mensaje: 80
Mensaje: 81
Mensaje: 82
Mensaje: 83
Mensaje: 84
Mensaje: 85
Mensaje: 86
Mensaje: 87
Mensaje: 88
Mensaje: 89
Mensaje: 90
Mensaje: 91
Mensaje: 92
Mensaje: 93
Mensaje: 94
Mensaje: 95
Mensaje: 96
Mensaje: 97
Mensaje: 98
Mensaje: 99
Mensaje: 100
Mensaje: 101
Mensaje: 102
Mensaje: 103
Mensaje: 104
Mensaje: 105
Mensaje: 106
Mensaje: 107
Mensaje: 108
Mensaje: 109
Mensaje: 110
Mensaje: 111
Mensaje: 112
Mensaje: 113
Mensaje: 114
```

Y luego si que mantiene el orden de envío



La conclusión a la que llego es que los primero 80 mensajes se han perdido